TP3 · Projet ML complet

Analyse de sentiment des commentaires YouTube

Du développement au déploiement cloud : modèle scikit-learn, API FastAPI, conteneur Docker sur GitHub Codespaces et extension Chrome « Écho ».

Accuracy	F1 macro	Latence p95 (50 commentaires)	Version du modèle
89.15 %	**0.882**	**1.5 ms**	**cdc1229a79f6**

Établissement	ENSAM Rabat · Université Mohammed V · Filière INDIA
Encadrante	Prof. Maria Zemzami
Auteur(s)	Rim Benamar
Année universitaire	2026/27
Date du rapport	06/11/2026

1. Analyse du problème et solution proposée

Problématique

Une vidéo YouTube peut recevoir des centaines, voire des milliers de commentaires. Les lire pour comprendre la réaction du public est chronophage, impossible à faire à grande échelle, subjectif (deux lecteurs ne retiennent pas les mêmes commentaires) et difficile à transformer en chiffre utilisable pour une décision.

Objectif

Classer automatiquement chaque commentaire en **négatif (-1)**, **neutre (0)** ou **positif (1)** et présenter, directement sur la page de la vidéo, une synthèse chiffrée : répartition, score global, liste filtrable.

Contraintes du cahier des charges

Critère	Exigence	Obtenu	Statut
Accuracy (test)	≥ 80 %	89.15 %	✔
F1 par classe	> 0,75	0.818 / 0.929 / 0.899	✔
Inférence, batch de 50	< 100 ms	p95 = 1.5 ms	✔
Dataset	≥ 1 000 lignes, ≥ 300 / classe	36 488 lignes	✔
API	REST, publique, HTTPS	FastAPI · Codespaces · port 7860	✔
Interface	Extension Chrome	Manifest V3 · service worker	✔

Solution

Le système suit une démarche MLOps de bout en bout : un pipeline de données reproductible, un modèle optimisé et versionné, une API REST conteneurisée exécutée dans le cloud, une extension Chrome et des tests automatisés à chaque niveau (nettoyage, modèle, API, interface). Chaque étape se relance par une commande unique (`python run_pipeline.py`, `bash scripts/start_api.sh`, `pytest`).

2. Architecture

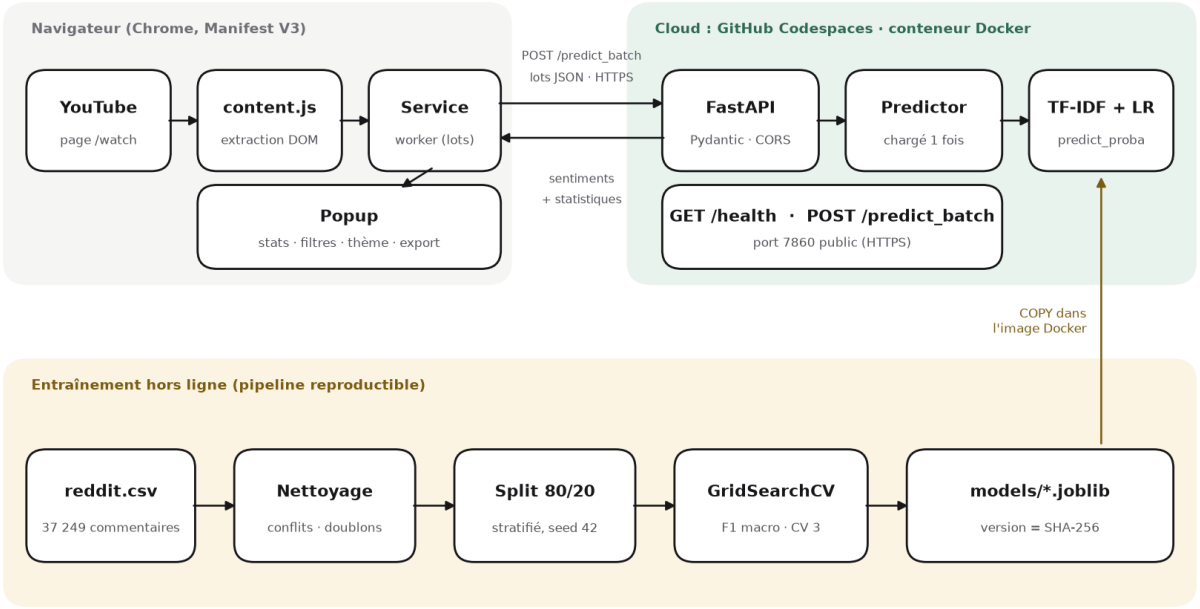


Figure 1 — Architecture : flux d'analyse en ligne (haut) et pipeline d'entraînement hors ligne (bas).

Flux de données

- **1–2.** Sur une page youtube.com/watch, le script de contenu fait défiler la section commentaires (chargée paresseusement par YouTube) puis lit le DOM.
- **3.** Le **service worker** découpe les textes en lots de 100 et les envoie à P0ST `/predict_batch` (3 requêtes en parallèle). Il tourne indépendamment du popup : fermer le popup n'interrompt pas l'analyse.
- **4.** L'API valide la requête (Pydantic), nettoie les textes avec la **même fonction qu'à l'entraînement**, vectorise (TF-IDF) et prédit les probabilités (régression logistique) en une seule opération vectorisée.
- **5–6.** Les prédictions et statistiques sont stockées par vidéo (`chrome.storage`) ; le popup les affiche et se met à jour en direct pendant l'analyse.

Composants

Couche	Technologie	Emplacement
Frontend	Extension Chrome MV3 (HTML, CSS, JS sans dépendance)	chrome-extension/
Backend	FastAPI, Pydantic v2, Uvicorn	src/api/, app_api.py
ML	scikit-learn : TF-IDF + LogisticRegression, joblib	src/models/, models/
Données	pandas, matplotlib	src/data/, reports/
Déploiement	Docker (python:3.10-slim), GitHub Codespaces, devcontainer	Dockerfile, .devcontainer/
Qualité	pytest, node --test, GitHub Actions	tests/, .github/workflows/

3. Données et préparation

Le dataset Reddit Sentiment Analysis (fichier `reddit.csv`) contient 37 249 commentaires labellisés -1 / 0 / 1. Il est téléchargé automatiquement dans `data/raw/` jamais modifié puis transformé vers `data/processed/`.

Nettoyage

- Décodage HTML, conversion des emojis fréquents en mots (cœur → love, larmes → sad, pouce baissé → bad), normalisation ASCII, minuscules.
- Suppression des URLs, mentions @, symboles # (le mot est gardé), horodatages « 3:45 », caractères spéciaux, lettres répétées (« soooo » → « soo »).
- Les négations (not, don't) sont **conservées** : elles inversent le sentiment.
- **Textes contradictoires** (même texte, labels différents) : tous retirés, plutôt que d'en garder un au hasard. Puis suppression des doublons.

Étape	Lignes
Fichier brut	37,249
Valeurs manquantes / labels invalides	− 100
Vides après nettoyage	− 214
Lignes contradictoires (1 textes)	− 3
Doublons	− 444
Jeu final	**36,488**
Train / test (80/20, stratifié, seed 42)	29,190 / 7,298

Analyse exploratoire

Répartition : positif 43.09 % (15 721) neutre 34.35 % (12 534) négatif 22.56 % (8 233). Le ratio de déséquilibre est de 1.91 : modéré. Plutôt que de sur-échantillonner la pondération `class_weight='balanced'` est testée dans la recherche d'hyperparamètres et la sélection se fait sur le F1 macro qui pénalise une classe minoritaire négligée. Les commentaires font en moyenne 29.8 mots (médiane 13 maximum 1 307).

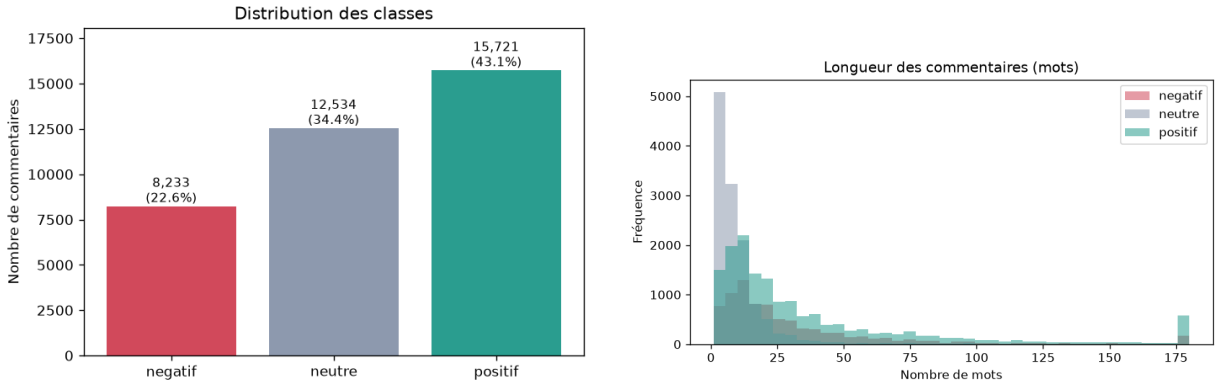


Figure 2 — Distribution des classes et longueur des commentaires.

4. Modélisation et choix techniques

Représentation : TF-IDF

Chaque commentaire devient un vecteur creux où le poids d'un mot est sa fréquence dans le commentaire (`sublinear_tf` : 1 + log tf) multipliée par l'inverse de sa fréquence dans le corpus. Les mots présents partout (the, video) pèsent peu ; les mots discriminants (awesome, worst) pèsent beaucoup. Contrairement à un simple comptage (CountVectorizer), on évite que les mots fréquents mais neutres dominent, et la normalisation L2 rend comparables les commentaires courts et longs.

Modèle : régression logistique

Modèle linéaire très adapté aux vecteurs creux de grande dimension : rapide à entraîner, inférence en quelques millisecondes, coefficients interprétables et surtout des **probabilités** exploitées comme indice de confiance dans l'interface.

Recherche d'hyperparamètres

GridSearchCV, validation croisée stratifiée à 3 plis sur le train uniquement, métrique F1 macro. Grille : n-grammes (1,1)/(1,2), taille du vocabulaire, C ∈ {1, 5, 10}, class_weight ∈ {None, balanced}. Meilleure combinaison : `C=10.0, class_weight=balanced, max_features=20000, ngram_range=[1, 1]`.

Rang	Paramètres	F1 macro (CV)	Écart-type
1	C=10.0, class_weight=balanced, vocab=50000, ngram=(1, 1)	0.8660	0.0026
2	C=10.0, class_weight=balanced, vocab=20000, ngram=(1, 1)	0.8660	0.0026
3	C=10.0, class_weight=None, vocab=20000, ngram=(1, 1)	0.8643	0.0021
4	C=10.0, class_weight=None, vocab=50000, ngram=(1, 1)	0.8643	0.0021
5	C=5.0, class_weight=balanced, vocab=20000, ngram=(1, 1)	0.8621	0.0010

Comparaison des algorithmes

Avec le même TF-IDF, la régression logistique est comparée à une **baseline** (classe majoritaire, la référence minimale à battre), un SVM linéaire calibré, un Naive Bayes multinomial et une Random Forest.

Modèle	Accuracy	F1 macro	F1 nég.	F1 neu.	F1 pos.	ms / 50
LogisticRegression ★	89.1 %	0.882	0.818	0.929	0.899	0.9
LinearSVC (calibré)	89.3 %	0.880	0.808	0.930	0.904	1.8
RandomForest	78.1 %	0.729	0.515	0.864	0.807	13.1
MultinomialNB	65.1 %	0.608	0.476	0.639	0.709	1.5
Baseline (majoritaire)	43.1 %	0.201	0.000	0.000	0.602	0.7

★ modèle retenu (meilleur F1 macro sur le test set ; la baseline est exclue de la sélection).

5. Résultats et performances

Sur les 7 298 commentaires du test set jamais vus pendant l'entraînement ni l'optimisation le modèle retenu (LogisticRegression) atteint une accuracy de **89.15 %** et un F1 macro de **0.882**.

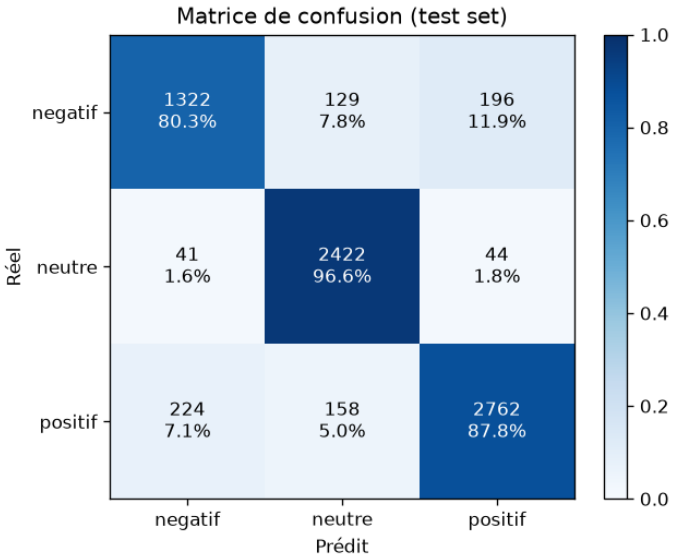


Figure 3 — Matrice de confusion (lignes : classe réelle, colonnes : classe prédite).

La diagonale porte l'essentiel des prédictions. La classe la plus difficile est **negatif** (F1 0.818) : c'est la minoritaire, et ses erreurs portent surtout sur la frontière avec le neutre (critiques implicites, ironie, phrases sans mot fortement polarisé). Les inversions négatif ↔ positif sont rares.

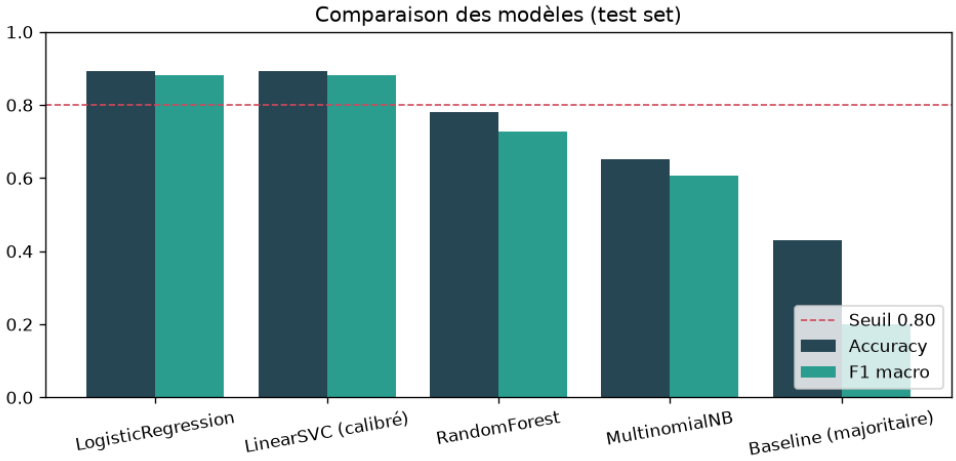


Figure 4 — Accuracy et F1 macro par modèle.

Latence

Taille du batch	p50	p95	max
1	0.36 ms	0.41 ms	0.64 ms
50	1.21 ms	1.94 ms	2.34 ms
250	3.48 ms	6.57 ms	9.43 ms
1000	12.19 ms	17.59 ms	20.63 ms

Nettoyage + vectorisation + prédiction sur modèle chargé (hors réseau). Le popup affiche en plus le temps mesuré côté API et le temps total ressenti.

Interprétabilité : mots les plus influents

Classe	Termes au poids le plus élevé
negatif	hate, stupid, bad, other, fuck, wrong, down, mean, evil, fucked, worst, fucking
neutre	cockroach, jobless, recap, chennai, proportions, photoshopped, massive, sharpen, respected, misery, arr, internal
positif	more, good, best, lol, great, right, many, win, own, most, better, kind

6. API REST

Endpoint	Méthode	Rôle
/health	GET	État de l'API, modèle chargé, version (empreinte SHA-256)
/predict_batch	POST	1 à 1 000 commentaires → sentiment, confiance, probabilités + statistiques
/predict	POST	Un seul commentaire
/docs	GET	Documentation Swagger générée automatiquement

Exemple

```
POST /predict_batch
{"comments": ["This video is amazing!", "Worst tutorial ever", "!!!"]}

200 OK
{"predictions": [{"text": "This video is amazing!", "sentiment": 1, "label": "positif",
                  "confidence": 0.93, "probabilities": {...}, "skipped": false}, ...],
 "statistics": {"total": 3, "analyzed": 2, "skipped": 1, "positive_pct": 50.0, ...},
 "processing_time_ms": 1.9, "model_version": "cdc1229a79f6"}
```

Choix de conception

- **FastAPI** plutôt que Flask : validation automatique (Pydantic), documentation interactive, serveur ASGI performant.
- **Chargement unique** du modèle au démarrage (lifespan) et pré-chauffe ; en cas d'échec l'API démarre quand même, `/health` renvoie « degraded » et les prédictions 503 : plus simple à diagnostiquer qu'un conteneur qui redémarre en boucle.
- **Validation** : batch vide, chaînes vides uniquement, mauvais type, plus de 1 000 éléments ou JSON invalide → 422 avec le champ fautif.
- **Textes vides après nettoyage** (« !!! ») marqués skipped et exclus des pourcentages, au lieu de gonfler artificiellement la classe neutre.
- **CORS** configuré (origines `chrome-extension://*`, liste paramétrable par `ALLOWED_ORIGINS`) ; en-tête `X-Process-Time-Ms` et journalisation de chaque requête.

7. Extension Chrome « Écho »

- **Extraction** : défilement automatique jusqu'au nombre demandé (arrêt si rien ne se charge 3 fois de suite), sélecteurs pour l'ancien et le nouveau DOM de YouTube, réponses optionnelles, injection à la demande si l'onglet précède l'installation.
- **Service worker** : l'analyse continue si le popup est fermé ; progression lot par lot visible en direct.
- **Onglet Aperçu** : jauge à aiguille du score global (-1 à +1) et verdict en mots (« Plutôt positif », « Avis partagés »…), part de chaque sentiment, commentaire le plus enthousiaste et le plus critique, temps de réponse et version du modèle.

- **Onglet Commentaires** : cartes avec un visage par sentiment et la confiance (3 probabilités au survol), filtres, recherche, tri par confiance, menu d'export.
- **Confort** : mode sombre/clair mémorisé, copie et export CSV/JSON des résultats filtrés, résultats conservés par vidéo, messages d'erreur explicites (URL, port privé, délai).

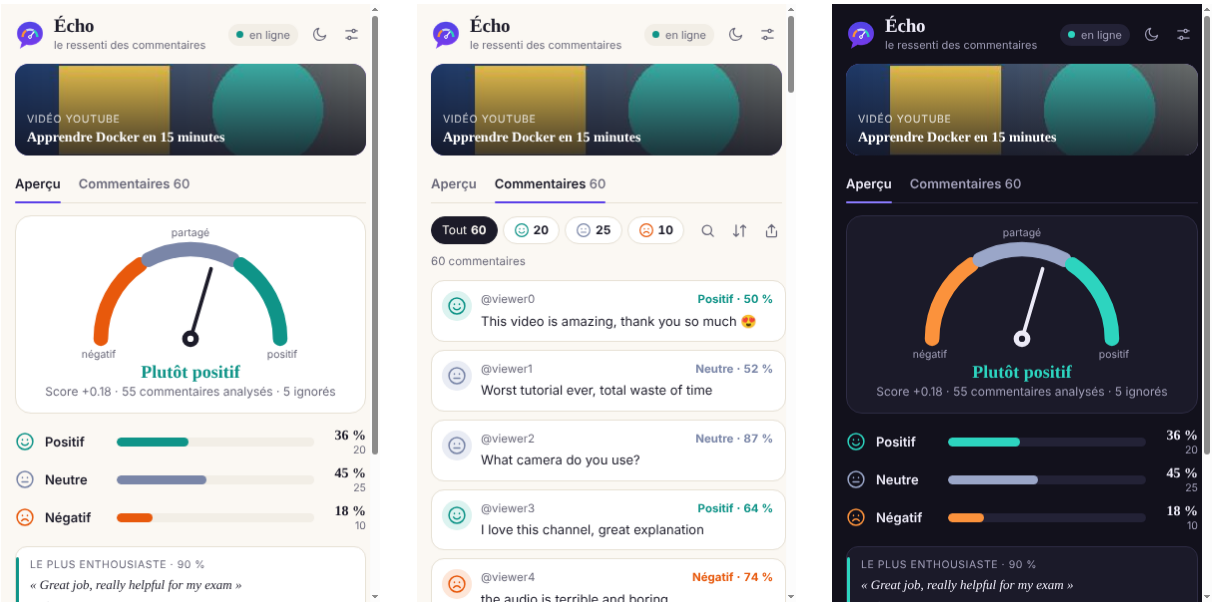


Figure 5 — Écho : onglet Aperçu, onglet Commentaires et mode sombre.

8. Déploiement et validation

Conteneur Docker

- Image `python:3.10-slim`, dépendances de production seulement (requirements.txt séparé de requirements-dev.txt).
- Couche des dépendances copiée avant le code : elle reste en cache tant que requirements.txt ne change pas.
- Seuls `src/api`, `src/utils`, `app_api.py` et `models/` sont copiés (.dockerignore) ; utilisateur non-root ; HEALTHCHECK sur /health ; port configurable par `PORT`.
- Versions de scikit-learn et joblib identiques à l'entraînement et en production (un .joblib n'est pas portable entre versions).

GitHub Codespaces

Hugging Face Spaces exigeant désormais un abonnement pour Docker, l'API s'exécute dans un Codespace. Le fichier `.devcontainer/devcontainer.json` installe Docker et lance `scripts/start_api.sh` **à chaque démarrage** : entraînement si le modèle manque, build, lancement du conteneur, passage du port 7860 en public et affichage de l'URL HTTPS. Après la mise en veille automatique (~30 min), rouvrir le Codespace suffit.

Éphémère ou permanent ?

Un Codespace est un environnement de développement : il s'arrête, son URL dépend du Codespace et il ne passe pas à l'échelle. Une mise en production réelle demanderait un hébergement permanent (Cloud Run, Render, Azure App Service…) avec URL stable, supervision, journaux centralisés, authentification et limitation de débit. Grâce à Docker, la même image s'y déploierait sans modification.

Tests

Niveau	Ce qui est vérifié
Nettoyage	URLs, mentions, emojis, négations, horodatages, textes vides et très longs
Modèle	Critères du cahier des charges, baseline battue, accuracy recalculée, cas limites (vide, emoji seul, 5 000 caractères, langues mélangées), latence
API	/health, batchs de 1 à 1 000, temps de réponse, 5 cas de données invalides, JSON malformé, textes ignorés, CORS ; aussi exécutables contre l'URL déployée (API_URL=…)
Extension	Fonctions pures (lots, statistiques, filtres, CSV, URLs) avec node --test
Intégration	GitHub Actions : tests, build de l'image, démarrage du conteneur et appels réels

9. Difficultés rencontrées et solutions

Difficulté	Solution
Commentaires YouTube chargés dynamiquement	Défilement automatique avec condition d'arrêt ; sélecteurs ancien et nouveau DOM
Popup fermé = analyse perdue	Analyse déplacée dans un service worker, état partagé via chrome.storage
Codespace en veille, port repassé en privé	devcontainer qui relance l'API au démarrage ; détection d'une réponse HTML (page de connexion) avec message explicite
Compilation de numpy sous Windows (Python récent)	Contraintes de version assouplies pour utiliser les roues précompilées
Parallélisme joblib/loky en échec sous Windows	Option --n-jobs et repli automatique sur un seul processus
Textes identiques avec labels opposés	Retrait des groupes contradictoires avant le split
LinearSVC sans probabilités	Calibration (CalibratedClassifierCV) pour comparer à armes égales

10. Limites et améliorations futures

- **Changement de domaine** : le modèle apprend sur Reddit et s'applique à YouTube (emojis, argot, commentaires très courts). Les scores ci-dessus valent pour Reddit ; annoter quelques centaines de commentaires YouTube permettrait de mesurer et réduire l'écart.
- **Langues** : le modèle est anglophone. Un Transformer multilingue (XLM-R, DistilBERT multilingue) quantifié couvrirait le français et la darija.
- **Sarcasme et contexte** : hors de portée d'un sac de mots ; mêmes pistes (Transformers) ou analyse par fil de discussion.
- **MLOps** : registre de modèles (MLflow), suivi de la dérive des prédictions, réentraînement planifié, déploiement continu de l'image.
- **Production** : hébergement permanent, clé d'API et limitation de débit, CORS restreint à l'identifiant de l'extension.

11. Conclusion

Le système répond à l'ensemble du cahier des charges : un modèle à 89.1 % d'accuracy et F1 > 0,75 sur chaque classe, une inférence en 1.5 ms (p95) pour 50 commentaires, une API conteneurisée accessible publiquement et une extension utilisable sur n'importe quelle vidéo. Au-delà du score, la démarche MLOps (données brutes immuables, pipeline reproductible, versions figées, modèle versionné par empreinte, conteneur, tests à chaque niveau) rend chaque résultat re-calculable et chaque déploiement répétable.